- Plan: FT-IR 云端 Web 客户端改造

Plan: FT-IR 云端 Web 客户端改造

目标是在保留现有 FT-IR 业务能力的前提下，将单机脚本/GUI 改造为可云端访问的 Web 客户端服务。推荐采用“Linux API 集群 + Windows RPA Worker 专机”的混合架构：并行化可计算步骤，串行化 OMNIC RPA 步骤，通过任务编排实现端到端稳定性与扩展性。

Steps

1. Phase 1 - 架构定稿与边界定义
2. 产出统一任务状态机（`queued -> preprocessing -> rpa_pending -> rpa_running -> postprocessing -> done/failed`），并定义失败重试与超时规则。
3. 拆分现有 `run_pipeline` 为三段可独立执行的服务函数：预处理段、RPA段、报告段（后续任务队列调用）。
4. 定义 API 边界：任务创建、任务详情、日志流、结果文件下载、历史任务查询。
5. Phase 2 - 后端服务化（可与 Phase 3 并行）
6. 新建 FastAPI 网关层，封装任务接口与鉴权；将文件上传落盘逻辑抽象为存储服务（本地/对象存储可切换）。
7. 引入任务队列与调度层（建议 Celery + Redis/RabbitMQ）：
   - `preprocess_queue` 并发 2-5
   - `rpa_queue` 并发 1（单 Windows Worker）
   - `postprocess_queue` 并发 5+
8. 将 pipeline 中硬编码路径改为任务上下文路径（job_id + stage），并统一使用跨平台路径拼装。
9. 将 stdout print 进度改为结构化日志事件（阶段、进度百分比、消息、时间戳），供 WebSocket/SSE 推送。
10. Phase 3 - Windows RPA Worker 改造（依赖 Phase 1，部分依赖 Phase 2）

11. 将 `software_agent/ir_rpa.py` 封装为独立 worker 可调用入口，输入为任务上下文（CSV 路径/下载地址），输出为标准产物路径与执行日志。
12. 增加 RPA 互斥锁与守护机制（同机仅1任务，异常时自动回收焦点/重启 OMNIC 流程）。
13. 增加心跳与健康检查 API，供调度系统判断 worker 可用性。
14. Phase 4 - Web 前端落地（可与 Phase 2 后半并行）
15. 前端技术栈建议：React + TypeScript + Vite + 组件库（如 Ant Design）+ ECharts（光谱交互图）。
16. 实现页面：任务提交页、实时进度页、任务详情页、报告预览页、历史列表页。
17. 通信策略：
    - HTTP REST：提交任务、拉取详情、下载文件
    - WebSocket（优先）或 SSE：实时日志与状态推送
18. 结果展示：直接渲染 HTML 报告、展示关键图谱图片、支持 CSV 曲线交互缩放。
19. Phase 5 - 安全、部署与可观测（依赖 Phase 2/3/4）
20. 鉴权与权限：API Token/JWT、任务隔离（按用户/项目命名空间）。
21. 云端部署：Linux 部署 API + 队列 + 对象存储；Windows 部署 RPA Worker（专机或独立 VM）。
22. 可观测：集中日志、任务追踪 ID、失败告警（RPA 卡死、模型超时、文件缺失）。
23. 性能策略：限制单用户并发、排队可视化、模型调用熔断与退避重试。
24. Phase 6 - 验证与灰度上线（依赖全部阶段）
25. 构建端到端回归样例集（使用 `Code/Demo` 中多种 CSV/图片）验证成功率与一致性。
26. 分别验证：无 RPA 路径（仅解析与报告）、含 RPA 路径（完整链路）、异常路径（模型超时/OMNIC 无响应）。
27. 灰度发布：先单租户内测，再扩大到多用户；持续监控队列堆积与平均处理时长。

Relevant files

- `c:/Users/34029/Desktop/IR-Project/Code/pipeline.py` — 核心编排入口；拆分为分段任务函数的首要参考。
- `c:/Users/34029/Desktop/IR-Project/Code/run_gui.py` — 现有日志输出与线程模型参考；迁移为服务端事件推送。
- `c:/Users/34029/Desktop/IR-Project/Code/software_agent/ir_rpa.py` — RPA 执行核心；独立 worker 化与互斥控制重点。
- `c:/Users/34029/Desktop/IR-Project/Code/report_generator/generator.py` — 报告合成与模型调用核心；需异步化并纳入 postprocess 队列。
- `c:/Users/34029/Desktop/IR-Project/Code/report_generator/template/report_template.html` — 前端报告预览结构参考。
- `c:/Users/34029/Desktop/IR-Project/Code/image_processing/extract.py` — 预处理/提取阶段可并行化核心。
- `c:/Users/34029/Desktop/IR-Project/Code/tests/test_basic.py` — 回归测试入口，可扩展 API 与任务流测试。

Verification

1. API 合约测试：提交任务、查询状态、下载报告、错误码一致性。
2. 队列验证：2-5 个并发请求下，`preprocess_queue` 并行执行，`rpa_queue` 串行不冲突。
3. RPA 稳定性验证：连续 20 个任务无人工干预完成率、失败可恢复率、平均耗时。
4. 前端体验验证：提交后 1 秒内出现任务记录，日志实时刷新，报告可预览/下载。
5. 资源与成本验证：CPU、内存、Windows worker 占用与模型 API 调用成本。

Decisions

- 已确认范围：Web 客户端优先、云端部署、支持 2-5 并发、允许依赖在线模型 API。
- 架构决策：采用分段异步队列；RPA 保持串行并部署在 Windows 专机。
- 包含范围：任务提交、进度可视化、结果展示与下载、基础鉴权与可观测。
- 排除范围（当前阶段）：移动端原生 App、完全离线模型替代、跨机构多租户复杂权限。

Further Considerations

1. 若必须将“完整处理并发”提升到 5（含 RPA），需规划 3-5 台 Windows Worker 池并引入远程桌面编排。

2. 建议尽早统一中间文件协议（对象存储 key 命名、元数据结构），避免后期迁移成本。
3. 建议在 MVP 阶段先保留当前 HTML 报告模板，第二阶段再引入可编辑报告工作流。